LivePose: Real-Time Multi-Camera 3D Pose Estimation

A Technical Report on Algorithms, Contributions, and Design Evolution

Shreekar Naik

May 2026

Contents

1 Introduction and Motivation **3**

2 System Overview **3**
2.1 Pipeline Overview . . . 3
2.2 Pose Detection Backend . . . 3

3 Detection Selection: Temporal Consistency at the 2D Level **4**
3.1 Problem . . . 4
3.2 Reprojection-Guided Selection . . . 4
3.3 Effect on Output . . . 4

4 Epipolar Consistency Filtering **4**
4.1 Motivation . . . 4
4.2 Fundamental Matrix . . . 5
4.3 Symmetric Epipolar Distance . . . 5
4.4 Camera Consistency Voting . . . 5
4.5 Effect on Output . . . 5

5 3D Triangulation: Confidence-Weighted DLT **5**
5.1 Linear Triangulation (DLT) . . . 5
5.2 Confidence Weighting . . . 6
5.3 Iterative Outlier Refinement . . . 6
5.4 Comparison to FreeMoCap . . . 6

6 Post-Triangulation Validation: Anthropometric Bounds **6**

7 Temporal Smoothing: Adaptive One-Euro Filter **7**
7.1 Why Not Kalman? . . . 7
7.2 Filter Mathematics . . . 7
7.3 Effect of Parameters . . . 8
7.4 Filter Lifecycle . . . 8

8 Bone Length Constraints **8**
8.1 Motivation . . . 8
8.2 Progressive Bone Model . . . 8
8.2.1 Online Update with Adaptive Learning Rate . . . 8
8.2.2 Compliance Schedule . . . 9
8.2.3 Corruption Detection . . . 9
8.3 XPBD Constraint Solving . . . 9

8.3.1 From FABRIK to XPBD . . . 9
8.3.2 Algorithm . . . 9
8.3.3 Effect on Output . . . 10
8.3.4 Influence of the Compliance Parameter . . . 10

9 Algorithmic Evolution **10**
9.1 Triangulation . . . 10
9.2 Temporal Filtering . . . 10
9.3 Bone Constraints . . . 11
9.4 Detection Selection . . . 11

10 Factor Analysis: How Each Component Affects the Output **11**
10.1 Cascade Effects . . . 11

11 Calibration **11**
11.1 Charuco Board . . . 11
11.2 Intrinsics . . . 11
11.3 Extrinsics . . . 12

12 Complete Pipeline Algorithm **12**

13 Contributions Summary **12**

14 Conclusion **12**

1 Introduction and Motivation

LivePose is a real-time multi-camera 3D human pose estimation system built from first principles, inspired by and benchmarked against the FreeMoCap project [1]. FreeMoCap demonstrated that markerless motion capture is feasible with consumer webcams and open-source pose detectors; however, its architecture is oriented towards *offline* batch processing of pre-recorded video sessions. LivePose extends this paradigm in two important directions:

1. **Live-first streaming**: The pipeline is designed around a continuous frame-streaming loop rather than a post-hoc processing step. Videos, when loaded, are treated as frame sources playing at their native FPS through the same path as live camera feeds.
2. **Robust online estimates**: Rather than accumulating many views of each pose and batch-refining, LivePose must produce a plausible 3D skeleton from each time step independently, using learned priors carried forward from previous frames.

The core algorithmic contributions over a naive FreeMoCap-style pipeline are:

- Confidence-weighted DLT triangulation with adaptive outlier refinement,
- Epipolar consistency voting to reject mis-detected camera views,
- Reprojection-guided temporal detection selection,
- Adaptive One-Euro filtering instead of fixed-gain Kalman smoothing,
- A progressive bone-length model with XPBD-based soft constraint solving.

2 System Overview

2.1 Pipeline Overview

LivePose processes synchronized multi-camera video streams through a sequence of stages that progressively refine the pose estimate. The system first detects 2D human keypoints independently in each camera view, then selects temporally consistent detections before applying geometric consistency checks across cameras. The surviving observations are triangulated into 3D joint positions, followed by temporal smoothing and biomechanical constraint enforcement to stabilize the final skeleton estimate.

The resulting 3D skeleton at time t is represented as $(\mathbf{X}_t, \mathbf{v}_t)$, where $\mathbf{X}_t \in \mathbb{R}^{J\times 3}$ contains the world-space coordinates of the $J = 17$ COCO joints and $\mathbf{v}_t \in \{0,1\}^J$ denotes joint validity.

2.2 Pose Detection Backend

The 2D detector is YOLO v8-pose [2], selected for its real-time throughput on CPU (nano variant) and GPU/MPS (small variant). The model is trained on COCO and outputs $J = 17$ keypoints per detected person.

Unlike FreeMoCap (which relies on MediaPipe Holistic), LivePose exposes a pluggable backend interface and supports MediaPipe and RTMPose as drop-in replacements. For all backends the interface contract is:

$$\text{process_batch}(\{I_k\}_{k=1}^{C}) \;\longrightarrow\; \big\{\{\hat{p}_n^{(k)}\}_{n=1}^{N_k}\big\}_{k=1}^{C} \tag{1}$$

where I_k is the frame from camera k, C is the number of cameras, N_k is the number of detected persons in camera k, and each detection $\hat{p}_n^{(k)}$ is a pair $(\mathbf{U}_n^{(k)}, \mathbf{c}_n^{(k)})$ with $\mathbf{U}_n^{(k)} \in \mathbb{R}^{J\times 2}$ (pixel coordinates) and $\mathbf{c}_n^{(k)} \in [0,1]^J$ (per-joint confidence scores).

Detections are sorted by descending mean confidence before being handed to the selection stage.

3 Detection Selection: Temporal Consistency at the 2D Level

3.1 Problem

A naïve approach takes the highest-confidence detection in each camera. This fails when multiple people are present or when a chair-back, bag, or background silhouette generates a spurious high-confidence detection. The error propagates immediately into the 3D estimate because all subsequent stages treat the selected detection as ground truth.

3.2 Reprojection-Guided Selection

LivePose maintains per-frame state:

$$\hat{\mathbf{X}}_{t-1} \in \mathbb{R}^{J\times 3} \quad \text{(last triangulated 3D skeleton)}$$
$$\hat{\mathbf{u}}_{t-1}^{(k)} \in \mathbb{R}^{2} \quad \text{(2D centroid in camera } k \text{ from previous frame)}$$

At time t, the reference point for camera k is chosen as follows:

1. **Reprojection prior** (preferred): If $\hat{\mathbf{X}}_{t-1}$ exists, project its valid joints into camera k via $\mathbf{P}_k$ and compute the centroid of all visible projections:

$$\mu_{\text{ref}}^{(k)} = \frac{1}{|\mathcal{V}|} \sum_{j\in\mathcal{V}} \pi_k\left(\hat{\mathbf{X}}_{t-1,j}\right)$$

   where $\pi_k(\mathbf{X}) = \mathbf{P}_k[X, Y, Z, 1]^\top / w$ is the projective map with projection matrix $\mathbf{P}_k = \mathbf{K}_k[\mathbf{R}_k \mid \mathbf{t}_k]$, and $\mathcal{V}$ is the set of valid joint indices.

2. **2D centroid prior** (fallback): Use $\hat{\mathbf{u}}_{t-1}^{(k)}$ from the previous frame's selected detection.

3. **Cold start**: Select the detection with highest mean confidence.

Given a reference $\mu_{\text{ref}}^{(k)}$, each candidate detection n is scored:

$$s_n = \left\| \hat{c}_n^{(k)} - \mu_{\text{ref}}^{(k)} \right\|_2 - 0.1 \cdot \bar{c}_n^{(k)} \cdot d_{\max} \tag{2}$$

where $\hat{c}_n^{(k)}$ is the centroid of detection n (averaged over joints with $c_j \geq 0.3$), $\bar{c}_n^{(k)}$ is its mean confidence, and $d_{\max} = 150\,\text{px}$ is the gate threshold. The second term provides a confidence tiebreaker while keeping distance dominant.

If $s_n > d_{\max}$ for all candidates, the camera is marked as occluded (no detection) for this frame, rather than accepting a potentially wrong person. This hard gate prevents identity switches from polluting the 3D estimate.

3.3 Effect on Output

This stage is the primary defence against *person identity switches*. Without it, a single camera switching to a background detection can corrupt the triangulated 3D position for the corresponding half of the skeleton. With it, identity is held stable across frames as long as the subject moves less than $d_{\max}$ between frames.

4 Epipolar Consistency Filtering

4.1 Motivation

Even after temporal selection, one camera may have its subject partially occluded and detect a different person. The epipolar geometry between calibrated cameras provides a purely geometric test: the projection of a real 3D point onto camera b must lie on the epipolar line defined by the corresponding point in camera a.

4.2 Fundamental Matrix

For two fully calibrated cameras (k, l) with intrinsics $\mathbf{K}_k$, $\mathbf{K}_l$ and extrinsics $(\mathbf{R}_k, \mathbf{t}_k)$, $(\mathbf{R}_l, \mathbf{t}_l)$, the relative pose from k to l is:

$$\mathbf{R}_{\text{rel}} = \mathbf{R}_l \mathbf{R}_k^\top \tag{3}$$

$$\mathbf{t}_{\text{rel}} = \mathbf{t}_l - \mathbf{R}_l \mathbf{R}_k^\top \mathbf{t}_k \tag{4}$$

The essential matrix is $\mathbf{E} = [\mathbf{t}_{\text{rel}}]_\times \mathbf{R}_{\text{rel}}$ where $[\cdot]_\times$ denotes the skew-symmetric cross-product matrix. The fundamental matrix is then:

$$\mathbf{F}_{kl} = \mathbf{K}_l^{-\top} \mathbf{E}\, \mathbf{K}_k^{-1} \tag{5}$$

normalised so that $F_{33} = 1$.

4.3 Symmetric Epipolar Distance

For corresponding pixel observations $\mathbf{u}_k \in \mathbb{R}^2$ and $\mathbf{u}_l \in \mathbb{R}^2$, the symmetric epipolar distance is:

$$d_{\text{epi}}(\mathbf{u}_k, \mathbf{u}_l) = \frac{1}{2}\left(\frac{|\tilde{\mathbf{u}}_l^\top \mathbf{F} \tilde{\mathbf{u}}_k|}{\|(\mathbf{F}\tilde{\mathbf{u}}_k)_{1:2}\|} + \frac{|\tilde{\mathbf{u}}_l^\top \mathbf{F} \tilde{\mathbf{u}}_k|}{\|(\mathbf{F}^\top \tilde{\mathbf{u}}_l)_{1:2}\|} \right) \tag{6}$$

where $\tilde{\mathbf{u}} = [u, v, 1]^\top$ is the homogeneous representation and the numerator is the algebraic epipolar residual.

4.4 Camera Consistency Voting

For each ordered pair (k, l) of cameras, the median symmetric epipolar distance is computed across all $J = 17$ joints:

$$d_{kl} = \text{median}_{j \in \mathcal{V}_{kl}}\, d_{\text{epi}}(\mathbf{u}_j^{(k)}, \mathbf{u}_j^{(l)})$$

where $\mathcal{V}_{kl}$ is the set of joints with valid (high-confidence) detections in both cameras. The score of camera k is its median across all pairs involving k:

$$D_k = \text{median}_{l \neq k}\, d_{kl}$$

Camera k is dropped from the frame if $D_k > \bar{D} + \tau_{\text{epi}}$ where $\bar{D}$ is the group median and $\tau_{\text{epi}} = 15\,\text{px}$ by default. At least two cameras are always retained.

4.5 Effect on Output

Epipolar filtering removes cameras that, despite passing the 2D temporal gate, are geometrically inconsistent with the majority. This is particularly important in cluttered scenes where the temporal proximity heuristic can be fooled by a second person who happens to be near where the tracked subject was in the previous frame. Removing such a camera before triangulation avoids systematic triangulation bias toward an incorrect detection.

5 3D Triangulation: Confidence-Weighted DLT

5.1 Linear Triangulation (DLT)

Given a 3D point $\mathbf{X} = [X, Y, Z]^\top$ and its projection $\mathbf{u} = [u, v]^\top$ in camera k with projection matrix $\mathbf{P}_k \in \mathbb{R}^{3\times 4}$, the projective constraint is:

$$\tilde{\mathbf{u}} \times \mathbf{P}_k \tilde{\mathbf{X}} = \mathbf{0}$$

Expanding the cross product yields two independent linear equations per camera view:

$$(u\,\mathbf{p}^{(k)3} - \mathbf{p}^{(k)1})\tilde{\mathbf{X}} = 0 \tag{7}$$

$$(v\,\mathbf{p}^{(k)3} - \mathbf{p}^{(k)2})\tilde{\mathbf{X}} = 0 \tag{8}$$

where $\mathbf{p}^{(k)i}$ is the i-th row of $\mathbf{P}_k$.

5.2 Confidence Weighting

For joint j observed in cameras $\mathcal{K}_j = \{k : c_j^{(k)} \geq \theta_c\}$ with $|\mathcal{K}_j| \geq 2$, the weighted system matrix is:

$$\mathbf{A}_j = \begin{bmatrix} c_j^{(k_1)}(u_j^{(k_1)}\,\mathbf{p}^{(k_1)3} - \mathbf{p}^{(k_1)1}) \\ c_j^{(k_1)}(v_j^{(k_1)}\,\mathbf{p}^{(k_1)3} - \mathbf{p}^{(k_1)2}) \\ \vdots \\ c_j^{(k_m)}(u_j^{(k_m)}\,\mathbf{p}^{(k_m)3} - \mathbf{p}^{(k_m)1}) \\ c_j^{(k_m)}(v_j^{(k_m)}\,\mathbf{p}^{(k_m)3} - \mathbf{p}^{(k_m)2}) \end{bmatrix} \in \mathbb{R}^{2m\times 4} \tag{9}$$

The homogeneous 3D point $\tilde{\mathbf{X}}_j \in \mathbb{R}^4$ is the right singular vector of $\mathbf{A}_j$ corresponding to its smallest singular value (solved via SVD). The Euclidean position is recovered by dehomogenisation: $\mathbf{X}_j = \tilde{X}_{1:3}/\tilde{X}_4$.

Weighting by confidence causes high-confidence observations to contribute stronger row norms in $\mathbf{A}_j$, pulling the solution toward them.

5.3 Iterative Outlier Refinement

When $|\mathcal{K}_j| > 2$, a second refinement pass rejects cameras with anomalously large reprojection errors. The reprojection error for camera k and joint j is:

$$\epsilon_j^{(k)} = \left\| \pi_k(\mathbf{X}_j) - \mathbf{u}_j^{(k)} \right\|_2$$

A camera is dropped if $\epsilon_j^{(k)} > \bar{\epsilon}_j + \sigma_{\text{out}} \cdot \hat{\sigma}_j$ where $\bar{\epsilon}_j$ and $\hat{\sigma}_j$ are the empirical mean and standard deviation of reprojection errors across cameras, and $\sigma_{\text{out}} = 2.0$. The system is then re-solved with the surviving cameras.

5.4 Comparison to FreeMoCap

FreeMoCap uses an unweighted DLT without refinement. By incorporating per-joint confidence as weights and performing the iterative outlier removal, LivePose reduces the influence of uncertain joints (e.g. occluded wrists) and corrupted views on the final 3D estimate.

6 Post-Triangulation Validation: Anthropometric Bounds

Before the triangulated skeleton enters the filtering and constraint stages, it is validated against hard anthropometric bounds. These bounds encode the known range of human bone lengths derived from biomechanics literature, expressed in millimetres for the COCO-17 joint topology (Table 1).

For a given frame, the *plausibility score* is:

$$s_{\text{plaus}} = \frac{\#\{\text{bones within bounds}\}}{\#\{\text{bones with both endpoints valid}\}} \tag{10}$$

If $s_{\text{plaus}} < 0.5$, the current frame's 3D estimate is discarded and the previous frame $\hat{\mathbf{X}}_{t-1}$ is used in its place (frame repetition). This prevents physically implausible skeletons from entering the smoothing and constraint stages, where they would corrupt the learned bone statistics.

Table 1: Anthropometric bone length bounds (COCO-17, in mm).

Joint pair (COCO indices)	Segment	Min (mm)	Max (mm)
(5, 7), (6, 8)	Upper arm	200	400
(7, 9), (8, 10)	Forearm	180	350
(11, 13), (12, 14)	Femur	350	550
(13, 15), (14, 16)	Tibia	300	500
(5, 6)	Shoulder width	280	500
(11, 12)	Hip width	200	380
(5, 11), (6, 12)	Lateral torso	400	650

7 Temporal Smoothing: Adaptive One-Euro Filter

7.1 Why Not Kalman?

An earlier version of the pipeline used a Kalman filter for post-triangulation smoothing. The Kalman filter exhibited two primary issues in this setting:

1. **Lag during fast motion**: The process noise Q must be set high enough to track rapid limb movements; with Q too low, the filter lags by multiple frames, distorting fast actions.
2. **Noise amplification at rest**: With Q high, the filter provides little smoothing when the subject is stationary, amplifying triangulation noise.

The One-Euro filter [3] resolves this tradeoff through speed-adaptive cutoff behaviour: the filter applies stronger smoothing at rest while remaining responsive during rapid motion.

7.2 Filter Mathematics

The One-Euro filter operates as a speed-adaptive first-order low-pass filter. For each scalar dimension x (the filter is applied independently to all $3J = 51$ scalar components of the skeleton), with a sample at time t of value x_t:

Step 1 — Velocity estimation. The raw finite-difference velocity is low-pass filtered with a fixed derivative cutoff $f_d = 1\,\text{Hz}$:

$$\dot{x}_t^{\text{raw}} = \frac{x_t - \hat{x}_{t-1}}{\Delta t} \tag{11}$$

$$\alpha_d = \frac{1}{1 + \tau_d/\Delta t}, \quad \tau_d = \frac{1}{2\pi f_d} \tag{12}$$

$$\hat{\dot{x}}_t = \alpha_d \dot{x}_t^{\text{raw}} + (1 - \alpha_d)\hat{\dot{x}}_{t-1} \tag{13}$$

Step 2 — Adaptive cutoff. The position cutoff frequency scales with estimated speed:

$$f_c(t) = f_{\min} + \beta\,|\hat{\dot{x}}_t| \tag{14}$$

where $f_{\min} = 3.0\,\text{Hz}$ (rest smoothing, configurable) and $\beta = 0.05$ (speed sensitivity, configurable).

Step 3 — Position filtering.

$$\alpha_c = \frac{1}{1 + \tau_c/\Delta t}, \quad \tau_c = \frac{1}{2\pi f_c(t)} \tag{15}$$

$$\hat{x}_t = \alpha_c x_t + (1 - \alpha_c)\hat{x}_{t-1} \tag{16}$$

7.3 Effect of Parameters

- $f_{\min}$ **(rest cutoff)**: Controls how much smoothing is applied to near-stationary joints. Higher $f_{\min}$ reduces lag but increases noise at rest.
- β **(speed coefficient)**: Controls how quickly the filter opens up during motion. Higher β tracks fast movements better but introduces more noise during transitions.

At $f_{\min} = 3\,\text{Hz}$ and $\beta = 0.05$, the filter provides $\approx$2 frames of smoothing at rest and near-zero lag for movements exceeding $\sim$60 mm/frame.

7.4 Filter Lifecycle

Each joint's filter is reset (re-initialised to the current observation) when the joint transitions from invalid to valid state, preventing transient artefacts where the filter attempts to smooth from its last valid position to the new observation.

8 Bone Length Constraints

8.1 Motivation

Triangulation errors and detector noise cause the inferred bone lengths to fluctuate even when the skeleton is not moving. Since human bone lengths are rigid (approximately constant), any fluctuation is pure error. Enforcing bone-length consistency both reduces noise and propagates information from well-observed joints to poorly-observed ones.

8.2 Progressive Bone Model

LivePose uses a subject-specific bone model that learns from observations rather than relying on population averages. For each bone b (there are $B = 16$ skeletal connections in COCO-17), the model maintains an online Gaussian estimate $\mathcal{N}(\mu_b, \sigma_b^2)$ of the true bone length, together with a confidence $\rho_b \in [0, 1]$ that determines how strongly the estimate is enforced.

8.2.1 Online Update with Adaptive Learning Rate

Bone length observations enter the model only when:

1. Both endpoint joints are valid ($v_a = v_b = 1$).
2. Both endpoints have reprojection error $< 15\,\text{px}$.
3. The measured length $\ell_b = \|\mathbf{X}_a - \mathbf{X}_b\|_2$ is within the anthropometric bounds from Table 1.

Given a clean observation ℓ_b, the update proceeds as follows. A Gaussian gate prevents large outliers from perturbing the model:

$$g_b = \exp\left(-\frac{1}{2}\left(\frac{\ell_b - \mu_b}{3\sigma_b}\right)^2\right) \tag{17}$$

(set to $g_b = 1$ until $n_b \geq 2$ observations have been collected). The effective learning rate decays with the number of observations:

$$\alpha_b = \text{clip}\left(\frac{\alpha_0 \cdot g_b}{1 + n_b \cdot \lambda},\ \alpha_{\min},\ 1\right) \tag{18}$$

with $\alpha_0 = 0.3$, $\lambda = 0.05$ (decay), and $\alpha_{\min} = 0.01$. The mean and variance are updated via Welford's online algorithm:

$$\mu_b \leftarrow (1 - \alpha_b)\mu_b + \alpha_b \ell_b \tag{19}$$

$$\sigma_b^2 \leftarrow (1 - \alpha_b)\big(\sigma_b^2 + \alpha_b(\ell_b - \mu_b')^2\big) \tag{20}$$

where μ_b' is the pre-update mean. The confidence ρ_b accumulates as $\rho_b \leftarrow \min(1.0,\ \rho_b + 0.05)$.

8.2.2 Compliance Schedule

The compliance γ_b (how loosely the constraint is enforced) follows a discrete schedule based on observation count n_b:

$$\gamma_b = \begin{cases} 0.8 & n_b < 10 \quad \text{(initialisation — highly flexible)} \\ 0.4 & 10 \le n_b < 30 \quad \text{(stabilisation)} \\ 0.1 & n_b \ge 30 \quad \text{(locked — rigid enforcement)} \end{cases} \tag{21}$$

High compliance early on ensures the model doesn't over-constrain before the subject's bone lengths are reliably estimated. As confidence increases, constraints tighten.

8.2.3 Corruption Detection

Before updating the model, a frame-level corruption check is performed. If more than 50% of observable bones deviate from their model means by more than $3\sigma_b$, the entire frame is skipped (neither the model nor the bone estimates are updated). This prevents a single badly-detected frame from corrupting the running statistics.

8.3 XPBD Constraint Solving

8.3.1 From FABRIK to XPBD

Initially, FABRIK (Forward And Backward Reaching Inverse Kinematics) [4] was considered for the IK step. FABRIK is simple and computationally efficient for open kinematic chains, but COCO-17 is a *branching* skeleton (multiple chains sharing a common root), and FABRIK does not natively handle closed-loop or branching structures without ad-hoc merging logic.

LivePose replaces FABRIK with an XPBD-style (extended Position-Based Dynamics) [5] constraint solver, which handles arbitrary constraint topologies uniformly and has a natural compliance parameter that maps directly onto the learned bone model.

8.3.2 Algorithm

The XPBD solver operates in the position domain. Let $\mathbf{p}_j \in \mathbb{R}^3$ denote the position of joint j (initialised to the smoothed triangulated position for valid joints). For each bone constraint (a, b) with target length ℓ_b^* and effective weight w_b, the following correction is applied at each iteration:

$$\delta_b = (\|\mathbf{p}_a - \mathbf{p}_b\|_2 - \ell_b^*) \cdot \frac{\mathbf{p}_a - \mathbf{p}_b}{\|\mathbf{p}_a - \mathbf{p}_b\|_2} \tag{22}$$

The effective weight w_b is computed from the compliance and stiffness of the bone:

$$w_b = 1 - \text{clip}\left(\frac{\gamma_b}{s_b},\ 0,\ 1\right) \tag{23}$$

where $s_b \in \{0.3, 0.7, 1.0\}$ is the bone stiffness (face/head: 0.3, limbs: 0.7, torso: 1.0). The position update depends on the validity of each endpoint:

$$\begin{cases} \mathbf{p}_a \leftarrow \mathbf{p}_a - \frac{1}{2} w_b \, \delta_b \\ \mathbf{p}_b \leftarrow \mathbf{p}_b + \frac{1}{2} w_b \, \delta_b \end{cases} \quad \text{if both } a, b \text{ valid} \tag{24}$$

$$\mathbf{p}_b \leftarrow \mathbf{p}_a - \frac{\mathbf{p}_a - \mathbf{p}_b}{\|\mathbf{p}_a - \mathbf{p}_b\|_2} \cdot \ell_b^* \quad \text{if only } a \text{ valid} \tag{25}$$

(symmetric case for only b valid). This is iterated $I = 5$ times per frame.

8.3.3 Effect on Output

For fully triangulated joints ($v_a = v_b = 1$), XPBD acts as a soft regulariser: the positions are nudged by a fraction w_b of the length error, distributing the correction symmetrically. For joints where one endpoint is invalid (occluded), the valid endpoint acts as an anchor and the invalid one is placed along the bone direction at the correct length. This effectively provides *kinematic extrapolation* from observed joints to occluded ones.

8.3.4 Influence of the Compliance Parameter

The compliance schedule from Equation (21) directly modulates how much of the bone length error is corrected per iteration. During the learning phase ($\gamma = 0.8$), only 20% of the error is applied per half-joint; in the locked phase ($\gamma = 0.1$ with torso stiffness $s = 1.0$), 90% of the error is corrected. The gradual tightening prevents large corrections during the observation phase, where the model mean μ_b may still be noisy.

9 Algorithmic Evolution

This section summarises the progression of the major algorithmic decisions during development.

9.1 Triangulation

The earliest implementation used a standard unweighted DLT triangulation approach similar to existing markerless motion capture systems. Confidence weighting was later introduced to leverage the per-joint confidence estimates produced by the detector. A reprojection-error refinement pass was subsequently added to reject inconsistent camera observations before the final triangulation solve.

9.2 Temporal Filtering

Phase 1 — No filtering. The initial system directly exposed raw triangulation output, which resulted in visible high-frequency jitter.

Phase 2 — Kalman filtering. A per-joint constant-velocity Kalman filter was introduced to stabilize the skeleton trajectories. Extended Kalman variants were also explored for direct multi-camera sensor fusion. Although these approaches improved smoothness, they required careful tuning of motion and measurement noise parameters and introduced noticeable lag during rapid movement.

Phase 3 — One-Euro filtering. The Kalman-based approach was replaced with the adaptive One-Euro filter, which provided a more practical balance between responsiveness and noise suppression under varying motion speeds.

9.3 Bone Constraints

Phase 1 — Hard projection. The earliest constraint stage projected the skeleton onto a rigid anthropometric model with fixed population-average bone lengths.

Phase 2 — Online statistical modelling. Bone lengths were later estimated online using running statistical estimates, allowing the model to adapt to the observed subject rather than relying on fixed human proportions.

Phase 3 — Progressive XPBD enforcement. The final system introduced the progressive bone model described in Section 7 together with an XPBD-based constraint solver. FABRIK was initially considered for inverse kinematics, but XPBD proved better suited to the branching structure of the COCO-17 skeleton and provided more controllable soft-constraint behaviour.

9.4 Detection Selection

The earliest detection stage selected the highest-confidence detection independently in each camera view. This produced frequent identity switches in cluttered scenes or multi-person environments. The later reprojection-guided selection stage and epipolar consistency filtering introduced both temporal and geometric consistency constraints, substantially improving identity stability across frames.

10 Factor Analysis: How Each Component Affects the Output

Table 2 summarises how each algorithmic component affects the quality of the output skeleton.

10.1 Cascade Effects

The stages are not independent. Detection selection errors at the 2D level propagate through the entire chain; a wrong detection in two cameras will produce a triangulation that is geometrically consistent (it passes the epipolar filter) but anatomically impossible (caught by the anthropometric gate). Conversely, a clean detection that produces a high-reprojection-error view (perhaps due to calibration error) is handled by the DLT refinement pass without the upstream stages needing to intervene.

11 Calibration

11.1 Charuco Board

Camera calibration uses a Charuco board (ChArUco = Chessboard + ArUco), which combines the robustness of ArUco markers (enabling corner identification even under partial occlusion) with the sub-pixel accuracy of chessboard corners.

11.2 Intrinsics

For each camera k, intrinsic calibration solves:

$$\mathbf{K}_k = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{d}_k \in \mathbb{R}^5 \quad \text{(distortion coefficients)} \tag{26}$$

using the OpenCV Charuco calibration routine, which minimises reprojection error over at least 15 board views per camera.

Table 2: Factor analysis: each algorithmic component and its failure mode when absent.

Component	**Error if absent**	**How modelled**	**Key parameter**
Confidence weighting in DLT	Occluded joints pull 3D position toward occluder	Confidence as row-weight in SVD	$\theta_c = 0.3$
Iterative DLT refinement	Single bad camera biases all joints	2σ reprojection outlier rejection	$\sigma_{\text{out}} = 2.0$
Temporal detection selection	Person identity switches corrupt skeleton	Distance to reprojected prior	$d_{\max} = 150\,\text{px}$
Epipolar filtering	Wrong-person detection in one camera warps 3D	Symmetric epipolar line distance	$\tau_{\text{epi}} = 15\,\text{px}$
Anthropometric gate	Physically impossible skeletons enter smoothing	Fraction of bones within bounds	Score ≥ 0.5
One-Euro filter	High-frequency triangulation noise (rest); lag (motion)	Adaptive cutoff $f_c = f_{\min} + \beta\lvert\dot{x}\rvert$	$f_{\min} = 3\,\text{Hz}$, $\beta = 0.05$
Bone model learning	Constraints based on wrong lengths distort posture	Online Welford with Gaussian gate	$\alpha_0 = 0.3$, $\lambda = 0.05$
XPBD IK	Bone lengths fluctuate; occluded joints not inferred	Soft compliance-weighted position correction	$I = 5$ iterations

11.3 Extrinsics

Extrinsic calibration estimates the relative pose between cameras by identifying shared board observations. For each synchronized frame containing a board visible to cameras k and l, the board-to-camera rotation and translation $(\mathbf{R}^{(k)}_{\text{board}}, \mathbf{t}^{(k)}_{\text{board}})$ are estimated via `cv2.solvePnP`. The relative camera pose is:

$$\mathbf{R}_{kl} = \mathbf{R}^{(l)}_{\text{board}}(\mathbf{R}^{(k)}_{\text{board}})^\top, \quad \mathbf{t}_{kl} = \mathbf{t}^{(l)}_{\text{board}} - \mathbf{R}_{kl}\mathbf{t}^{(k)}_{\text{board}}$$

Multiple shared observations are averaged: rotations via quaternion SLERP (with hemisphere alignment to avoid antipodal averaging artefacts), translations via arithmetic mean. A minimum of 8 shared frames per camera pair is required.

12 Complete Pipeline Algorithm

13 Contributions Summary

Table 3 summarises the key contributions over a baseline FreeMoCap-style offline pipeline.

14 Conclusion

LivePose demonstrates that robust online 3D pose estimation from multiple consumer cameras is achievable by layering complementary consistency constraints at each stage of the pipeline. The key

Table 3: Contributions relative to FreeMoCap-style baseline.

Component	**Baseline (FreeMoCap)**	**LivePose Contribution**
Triangulation	Unweighted DLT	Confidence-weighted DLT + reprojection-based outlier refinement
Detection	Highest-confidence selection	Reprojection-guided temporal selection with distance gate
Cross-camera validation	None	Epipolar consistency voting
Temporal filtering	None (batch)	Adaptive One-Euro filter (speed-adaptive cutoff)
Bone constraints	None	Progressive online bone model + XPBD soft-constraint IK
Validation	None	Anthropometric hard-bounds gate, frame rejection
Architecture	Offline batch	Live streaming pipeline with per-frame state

insight is that no single constraint is sufficient: epipolar geometry catches wrong detections that temporal proximity misses; anthropometric bounds catch physically impossible triangulations that the epipolar filter does not flag; and the XPBD solver enforces bone-length consistency without requiring a fixed human model, adapting instead to the specific subject observed.

The progression from the initial unweighted DLT through Kalman filtering to the One-Euro + XPBD combination illustrates a general principle: data-adaptive methods (speed-adaptive cutoff, online bone learning, compliance scheduling) outperform fixed parameter methods when the signal statistics are non-stationary, as is inevitable in real-world motion capture.

References

[1] Matthis, J. S. et al. (2023). *FreeMoCap: A free, open-source markerless motion capture system.* bioRxiv.

[2] Jocher, G. et al. (2023). *Ultralytics YOLOv8.* https://github.com/ultralytics/ultralytics.

[3] Casiez, G., Roussel, N., & Vogel, D. (2012). *1€ Filter: A simple speed-based low-pass filter for noisy input in interactive systems.* Proceedings of CHI 2012, pp. 2527–2530.

[4] Aristidou, A., & Lasenby, J. (2011). *FABRIK: A fast, iterative solver for the inverse kinematics problem.* Graphical Models, 73(5), 243–260.

[5] Macklin, M., Müller, M., & Chentanez, N. (2016). *XPBD: Position-based simulation of compliant constrained dynamics.* Proceedings of MIG 2016, pp. 49–54.

Algorithm 1 LivePose: Per-Frame 3D Skeleton Estimation

Require: Cameras $\{C_k\}_{k=1}^{K}$, frames $\{I_k\}_{k=1}^{K}$, prior state $(\hat{\mathbf{X}}_{t-1}, \hat{\mathbf{v}}_{t-1})$
Ensure: 3D skeleton $(\mathbf{X}_t, \mathbf{v}_t)$ and updated state

// **Stage 1: 2D Detection**
$\{\{\hat{p}_n^{(k)}\}\}_{k=1}^{K} \leftarrow \textsc{YoloBatch}(\{I_k\})$
// **Stage 2: Temporal Detection Selection**
for $k = 1$ to K **do**
  Compute reference $\mu_{\text{ref}}^{(k)}$ from $\hat{\mathbf{X}}_{t-1}$ or $\hat{\mathbf{u}}_{t-1}^{(k)}$
  Score each detection n via Eq. (2)
  $p^{(k)} \leftarrow \arg\min_n s_n$ if $s_n^* \leq d_{\max}$ else $\varnothing$
// **Stage 3: Epipolar Consistency**
Compute $\mathbf{F}_{kl}$ for all pairs via Eq. (5)
$\mathcal{K}_{\text{valid}} \leftarrow \textsc{EpipolarVoting}(\{p^{(k)}\}, \tau_{\text{epi}} = 15\,\text{px})$
// **Stage 4: Triangulation**
for $j = 1$ to J **do**
  Collect cameras $\mathcal{K}_j = \{k \in \mathcal{K}_{\text{valid}} : c_j^{(k)} \geq 0.3\}$
  if $|\mathcal{K}_j| \geq 2$ **then**
    Build $\mathbf{A}_j$ via Eq. (9); solve $\mathbf{X}_j \leftarrow \text{SVD}(\mathbf{A}_j)$
    [Refine] Reject cameras with $\epsilon_j^{(k)} > \bar{\epsilon}_j + 2\hat{\sigma}_j$; re-solve
    $v_j \leftarrow 1$
  else
    $v_j \leftarrow 0$
// **Stage 5: Anthropometric Gate**
$s_{\text{plaus}} \leftarrow \textsc{CheckBounds}(\mathbf{X}_t, \mathbf{v}_t)$
if $s_{\text{plaus}} < 0.5$ **then**
  $\mathbf{X}_t \leftarrow \hat{\mathbf{X}}_{t-1}$; **goto** Stage 8
// **Stage 6: Bone Model Update**
$\textsc{BoneModel.Observe}(\mathbf{X}_t, \mathbf{v}_t)$ ▷ Welford update, Eq. (20)
// **Stage 7: One-Euro Smoothing**
$\mathbf{X}_t \leftarrow \textsc{OneEuro}(\mathbf{X}_t, \mathbf{v}_t, \Delta t)$ ▷ Eq. (14)
// **Stage 8: XPBD Bone Constraint Solving**
if $\textsc{BoneModel.IsReady}$ **then**
  for $i = 1$ to $I = 5$ **do**
    for each bone (a, b) **do**
      Compute δ_b via Eq. (22); apply Eq. (24) or (25)
$\hat{\mathbf{X}}_t \leftarrow \mathbf{X}_t$; update centroids $\hat{\mathbf{u}}_t^{(k)}$
return $(\mathbf{X}_t, \mathbf{v}_t)$